

# Deep Reinforcement Learning

**Reward Engineering and Learning Rate Optimization on Pendulum-v1**

**Noah Van Leemput**

ACS-301  
Data & AI 6  
2025-2026

## Introduction:

This project investigates the application of Deep Reinforcement Learning (DRL) algorithms using Stable Baselines 3 on the Pendulum-v1 environment. The objectives are threefold: establish a baseline agent trained with the default reward function, improve agent behaviour through reward engineering, and analyse the impact of an additional extension in the form of hyperparameter tuning. Performance is evaluated using multiple training runs, TensorBoard logging, and deterministic evaluation to ensure robustness and reproducibility.

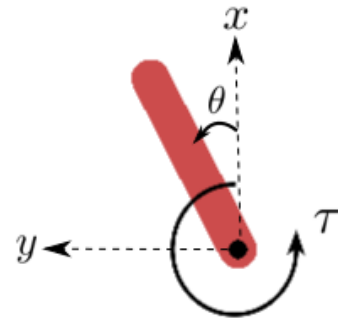
## Methodology:

### DRL Algorithm.

We use Proximal Policy Optimization (PPO), an on-policy actor-critic algorithm known for stable training in continuous control tasks. PPO optimizes a clipped surrogate objective to limit large policy updates and improve convergence stability.

### Environment.

Pendulum-v1 is a continuous control problem where the goal is to swing and stabilize a pendulum in the upright position. Episodes have a fixed length of 200 steps.



### Default Reward

The environment's default reward penalizes angular deviation, angular velocity, and control effort:

$$r = -(\theta^2 + 0.1\dot{\theta}^2 + 0.001\text{torque}^2)$$

- Theta is the pendulum's angle normalized between  $[-\pi, \pi]$ , zero is the upright position.
- Theta\_dot is the angular velocity
- Tau/Torque is the mechanical force applied in Nm. Positive counter-clock
- Based on the above equation, the minimum reward that can be obtained is:  $-(\pi^2 + 0.1 * 8^2 + 0.001 * 2^2) = -16.2736044$ , while the maximum reward is zero (pendulum is upright with zero velocity and no torque applied).
- The starting state is a random angle in  $[-\pi, \pi]$  and a random angular velocity in  $[-1, 1]$ .

## Custom Reward Design:

To better align the reward with the desired behaviour, we design a custom reward:

$$r = \cos(\theta) - 0.1\dot{\theta}^2$$

This formulation directly rewards upright positions while discouraging oscillations, resulting in smoother stabilization and improved learning dynamics.

## Extension: Hyperparameter Tuning (Learning Rate)

As an extension, we tune the PPO learning rate, evaluating multiple values to analyse their effect on convergence speed and stability.

## Results and Discussion

All results are evaluated using the mean episode return, averaged over complete 200-step episodes, ensuring consistent comparison across experiments.

### Baseline Performance

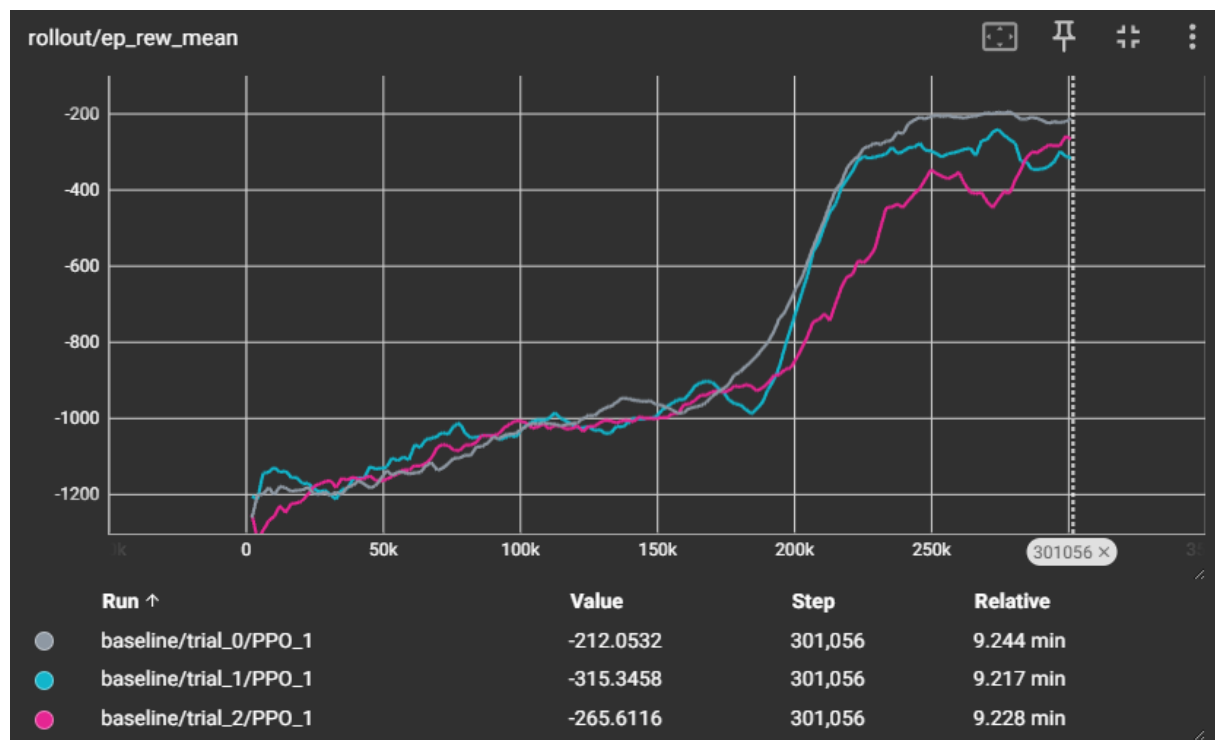


Figure 1: Training curves of PPO on Pendulum-v1 using the default reward over three random seeds.

While the agent can learn the task, convergence is slow and final performance varies significantly between trials, with final mean episode returns ranging from approximately -212 to -315. This indicates sensitivity to random initialization and less stable learning dynamics.

## Effect of Reward Engineering

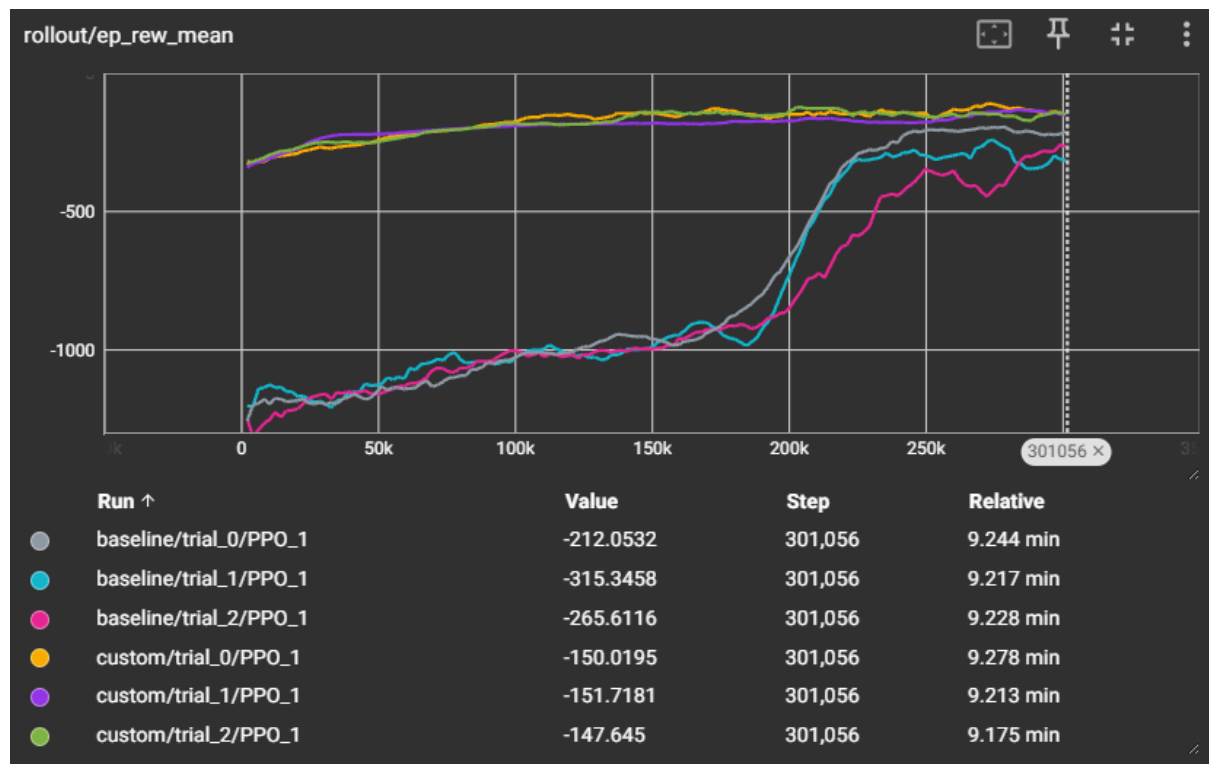


Figure 2: Training curves of PPO on Pendulum-v1 using a custom reward over three random seeds, compared to the curves of Figure 1.

Because the baseline and custom reward functions have different objectives, reward values must be interpreted with care. The default Pendulum-v1 reward strongly penalizes angle deviation, angular velocity, and torque, whereas the custom reward places greater emphasis on maintaining upright stability. As a result, relative performance trends such as convergence speed, final stability, and variance across runs provide a more meaningful basis for comparison than raw reward magnitude alone.

In contrast to the baseline, the agent trained with the custom reward converges substantially faster and consistently achieves higher episode returns across all runs. Final performance improves to mean episode returns of approximately  $-150$  for all trials, compared to a wider range between  $-212$  and  $-315$  for the baseline. Moreover, variance between runs is significantly reduced, indicating more stable behaviour and smoother control.

## Extension Analysis

As an extension, we experimented with changes to the learning rate parameter. Three sensible learning rates were evaluated: 0.0001, 0.0003 (the Stable Baselines 3 default), and 0.001.

Each configuration was trained for the same number of steps as the baseline (300,000), using the same environment and evaluation protocol as the baseline. One run per configuration was performed.

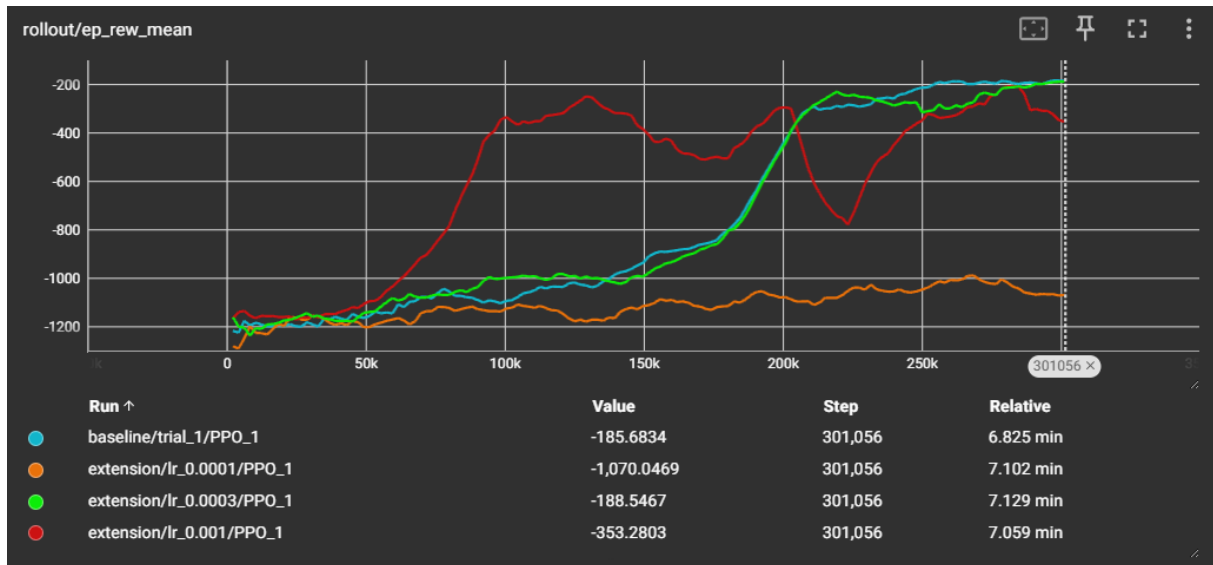


Figure 3: One of the 3 baseline trial curves from Figure 1 compared to 3 different hyperparameters.

The learning rate has a significant impact on PPO performance. When the learning rate is set too low (0.0001), learning progresses very slowly, with barely improved results at the end of the trial. In contrast, a higher learning rate (0.001), leads to a fast improvement in learning. However, this fast improvement is paired with unstable later performance, ending in a result significantly lower than the baseline.

The default learning rate (0.0003) yields the best overall performance, achieving returns close to identical to the baseline. This confirms that PPO's default configuration in Stable Baselines 3 is well-suited for the Pendulum-v1 task and that deviations from this value can negatively affect learning.

These results show the sensitivity of PPO to aggressive changes in learning rate and demonstrate that careful tuning is essential for achieving optimal performance, even when using established algorithms.

## Challenges

The default Pendulum-v1 reward strongly penalizes angle and velocity errors, resulting in lower absolute returns compared to the custom reward, which emphasizes upright stability via a cosine-based rule. To avoid misleading comparisons, performance was evaluated using relative improvements, convergence behaviour, and variance across runs rather than absolute reward returns alone.

Another challenge was the high variability observed between baseline training runs, where final episode returns differed noticeably across random seeds. This was addressed by conducting three independent training runs for both the baseline and custom reward settings, logging all experiments with TensorBoard, and focusing analysis on consistent trends rather than individual outliers. Deterministic evaluation was used throughout to ensure reproducibility and fair comparison.

## Conclusion

A key challenge was comparing performance across reward functions that differ in structure and emphasis, requiring evaluation based on convergence behaviour and stability rather than absolute reward values alone.

Learning rate tuning further highlights the sensitivity of PPO to learning rate selection. While the default learning rate performs robustly, overly small values slow learning considerably, and larger values introduce unstable shifts that degrade final performance. These results confirm that PPO's default configuration in Stable Baselines 3 is well suited for this task, but also emphasize the importance of careful tuning when deviating from established settings. Future work could extend this analysis by evaluating alternative algorithms such as SAC or by incorporating more advanced exploration strategies.